



ALBUKHARY INTERNATIONAL UNIVERSITY

BACHELOR IN COMPUTER SCIENCE (HONOURS)
BACHELOR IN DATA SCIENCE (HONOURS)

3. Python Built-in Data Types

In Python, there are four important built-in collection data types that allow you to store groups of related values. These are:

- List
- Tuple
- Set
- Dictionary

Each has its own characteristics and is used in different scenarios.

3.1. Lists

3.1.1. What is a list?

A list is an ordered, mutable (changeable) collection that can contain any type of data. Lists are written with square brackets [].

```
fruits = ["apple", "banana", "cherry"]
```

3.1.2. Key Features

- Items have a defined order and maintain the order you insert them.
- Items can be added, changed, or removed.
- Items with the same value can appear more than once

3.1.3. When to use lists

- An ordered collection:

```
ordered_list = ["first", "second", "third"]  
print(ordered_list[0]) # Output: first
```

- Modify the collection (add, update, delete items):

```
tasks = ["Read", "Write"]  
tasks.append("Sleep")  
print(tasks)
```

- Have duplicates:

```
numbers = [1, 1, 2, 3, 3]
```

- Example Use Case: Keeping a list of student names, a to-do list, shopping cart items.

```
my_list = [1, 2, 3]
my_list.append(4)
my_list.insert(1, 10)
my_list[0] = 99
my_list.remove(10)
sliced = my_list[1:3]
```

3.1.4. Nested list example

A list can contain other lists as its elements.

```
matrix = [[1, 2], [3, 4]]
print(matrix[1][0]) # Output: 3
```

This is useful for representing tables, grids, or matrices.

3.1.5. Creating tables with lists

You can represent tabular data using **nested lists**, where each inner list is a row of the table.

```
table = [
    ["Name", "Age", "Course"],
    ["Ali", 20, "AI"],
    ["Sara", 22, "Data Science"],
    ["John", 21, "Cybersecurity"]
]

for row in table:
    print(row)
```

3.2. Tuples

3.2.1. What is a Tuple?

A tuple is an ordered, immutable (unchangeable) collection. Tuples are written with parentheses ().

```
coordinates = (10.0, 20.0)
```

3.2.2. Key Features

- Items are in a specific order, just like lists.

- Once created, the values inside a tuple cannot be changed. You cannot add, remove, or modify elements.
- Value can be repeated.

3.2.3. When to Use Tuples

- An ordered collection (like a list):

```
exam_scores = (90, 85, 88)
print(exam_scores[0]) # Output: 90
```

- When you want to make sure the data remains constant.

```
config = ("localhost", 8080)
# Prevents accidental changes
```

- Tuples are faster than lists and use less memory.
- Example Use Case: Storing GPS coordinates, database records, fixed configuration.

3.2.4. Common Operation

```
tuple1 = (1, 2, 3)
print(tuple1[1])

#output 2
```

3.2.5. Tuple Unpacking

```
a, b = (5, 10)
print(a)
print(b)
```

3.2.6. Single Element Tuple

parentheses () alone do not define a tuple

```
single = ("hello",) # Must include a comma
```

Code	Result Type
<code>x = ("hello")</code>	<code><class 'str'></code>
<code>x = ("hello",)</code>	<code><class 'tuple'></code>

3.2.7. Example Tuple

- Database Record

A tuple is ideal for storing fixed records, such as database entries where data should not change (e.g., an employee record with name, ID, and department).

```
employee_record = ("John Doe", 101, "Engineering")
print(employee_record)

# Output: ('John Doe', 101, 'Engineering')
```

3.3. Sets

3.3.1. What is a Set?

A set is an unordered, mutable (Changeable) collection of unique elements. Sets are written with curly braces {}.

```
my_set = {1, 2, 3, 2}
print(my_set) # Output: {1, 2, 3}
```

3.3.2. Key Features

- Unordered: No defined order.
- Mutable: Items can be added or removed.
- No Duplicates: All elements are unique.

3.3.3. When to Use Sets

- To store unique values:

```
student_ids = {101, 102, 103, 101}
# Duplicate 101 is ignored
print(student_ids) # Output: {101, 102, 103}
```

- The order of elements is not important

```
colors = {"red", "blue", "green"}
```

3.3.4. Common Operations

Removing duplicates, checking common elements, storing unique tags or user roles.

```
s = {"apple", "banana"}
s.add("cherry")
s.remove("banana")
print("apple" in s)
```

3.3.5. Set Operations

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A.union(B))
print(A.intersection(B))
```

3.3.6. Real-World Use Case: Unique Values in a Database

Imagine you want to store unique user IDs that ensure no duplicates are allowed, such as a list of unique customer IDs for a website.

```
user_ids = {101, 102, 103, 104, 101}
# Duplicates are ignored
print(user_ids) # Output: {101, 102, 103, 104}
```

Sets are useful for:

- Eliminating duplicates.
- Performing quick membership testing (i.e., checking if an item exists in a collection).

3.3.7. Example of for loop and if statement

```
numbers = [1, 2, 2, 3, 4, 4, 5]      # Original list
with duplicates
unique_numbers = [ ]                # Empty list to
store unique values

for num in numbers:                  # Loop through
each number in the list
    if num not in unique_numbers:    # Check if the
number is not already in the unique list
        unique_numbers.append(num)  # Add the number
if it is unique

print(unique_numbers)

#Output: [1, 2, 3, 4, 5]
```

3.4. Dictionaries

3.4.1. What is a dictionary?

A dictionary is an unordered, mutable collection of key-value pairs. Dictionaries are written with curly braces {}.

```
student = {"name": "Ali", "age": 20}
```

3.4.2. Key Features

- Each item is stored as a pair where keys must be unique.
- No duplicate keys allowed.
- Can add, change, or remove items.

3.4.3. When to Use Dictionaries

- You need to associate keys with values:

```
person = {"name": "John", "age": 25} # 'name' and  
'age' are keys  
print(person["name"]) # Output: John
```

- You want fast access by key:

```
scores = {"math": 90, "science": 80}  
print(scores["science"])
```

- You are managing structured data: Key-value makes it easy to organize and look up data.

3.4.4. Common Operations

Storing user profiles, configuration settings, translation pairs.

```
info = {"name": "Zara", "course": "AI"}  
print(info["name"])  
info["course"] = "Data Science"  
info["grade"] = "A"  
info.pop("name")
```

3.4.5. Accessing Key, Value, Item

```
info.keys()  
info.values()  
info.items()
```

3.4.6. Example: Translation Pairs Example:

Dictionaries are great for mapping keys to values, such as storing translations for words.

```
translations = {  
    "hello": "hola",
```

```

    "goodbye": "adiós",
    "thank you": "gracias"
}

print(translations["hello"]) # Output: "hola"

```

Feature	List	Tuple	Set
Syntax	<code>my_list = ["apple", "banana", "cherry"]</code>	<code>my_tuple = ("apple", "banana", "cherry")</code>	<code>my_set = {"apple", "banana", "cherry"}</code>
Ordered	Maintains order of elements	Fixed order of elements	Order is not guaranteed
Changeable	<code>my_list[1] = "orange"</code>	Must convert to list to modify	<code>my_set.add("orange")</code>
Allows Duplicates	<code>["apple", "apple"]</code> is valid	<code>("apple", "apple")</code> is valid	Duplicates auto-removed
Indexing	<code>my_list[0] → "apple"</code>	<code>my_tuple[0] → "apple"</code>	Can't access via index
Use Case	Dynamic lists like a to-do list or cart	Fixed data like coordinates	Unique data like tags, categories
Common Methods	<code>.append()</code> , <code>.insert()</code> , <code>.remove()</code>	<code>.count()</code> , <code>.index()</code>	<code>.add()</code> , <code>.remove()</code> , <code>.union()</code>